

Fieldwork Processing

Outline



**VIDEO
CONCATENATION**



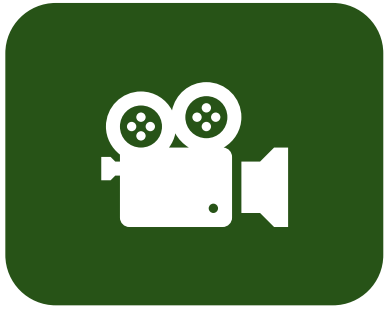
**FRAME
EXTRACTION**



GEOTAGGING



CLASSIFICATION



VIDEO CONCATENATION

```
Code Blame 47 lines (41 loc) · 1.61 KB
1 import argparse
2 import os
3 import subprocess
4 import glob
5
6 def concatenate_ffmpeg_folder(input_folder, output_path):
7     """
8     Concatenates all .mp4 files in a folder using ffmpeg (no re-encoding).
9     Assumes all input files are compatible (same codec, resolution, framerate).
10    """
11    # Get sorted list of all .mp4 files in the folder
12    clips = sorted(glob.glob(os.path.join(input_folder, "*.MP4")))
13    if len(clips) < 2:
14        raise ValueError("Need at least two .mp4 files to concatenate.")
15
16    # Create temporary file list for ffmpeg
17    file_list_path = os.path.join(input_folder, "concat_list.txt")
18    with open(file_list_path, "w") as f:
19        for clip in clips:
20            f.write(f"file '{os.path.abspath(clip)}'\n")
21
22    # Build ffmpeg command
23    command = [
24        "ffmpeg",
25        "-f", "concat",
26        "-safe", "0",
27        "-i", file_list_path,
28        "-c", "copy",
29        output_path
30    ]
31
32    try:
```

Concatenate GoPro Video
using python script. GoPro
uses this file naming
convention:

GHzzxxxxx.mp4

GXzzxxxxx.mp4

- “H” = AVC encoding
- “X” = HEVC encoding
- ‘xxxx’ = file number
- ‘zz’ = chapter number



FRAME EXTRACTION

Frames are then extracted from the concatenated video using python script. The date and time of photos will serve as the filename.

```
Code Blame 90 lines (71 loc) · 3.2 KB
1  import os
2  import cv2
3  import PIL
4  import time
5  import glob
6  import json
7  import pytz
8  import argparse
9  import subprocess
10
11  import numpy as np
12  import pandas as pd
13  import xml.etree.ElementTree as ET
14
15  from PIL import Image
16  from datetime import datetime, timedelta, timezone
17
18  # Extract start time using Exiftool
19  def extract_start_time(video_file):
20      """Extract the start time from a GoPro video file using ExifTool."""
21      try:
22          exiftool_path = './exiftool/exiftool-12.92_64/exiftool.exe'
23          exiftool_command = [exiftool_path, "-CreateDate", "-j", video_file]
24          result = subprocess.run(exiftool_command, stdout=subprocess.PIPE,
25                                 stderr=subprocess.PIPE, text=True, check=True)
26
27          metadata = json.loads(result.stdout)[0]
28          create_date_str = metadata.get('CreateDate')
29
30          if not create_date_str:
31              raise ValueError("CreateDate not found in video metadata")
32
```



FRAME EXTRACTION



frame_2025-05-14_09-01-49.jpg



frame_2025-05-14_09-01-50.jpg



frame_2025-05-14_09-01-51.jpg



frame_2025-05-14_09-01-52.jpg



frame_2025-05-14_09-01-53.jpg



frame_2025-05-14_09-01-54.jpg



frame_2025-05-14_09-01-55.jpg



frame_2025-05-14_09-01-56.jpg



frame_2025-05-14_09-01-57.jpg



frame_2025-05-14_09-01-58.jpg



frame_2025-05-14_09-01-59.jpg



frame_2025-05-14_09-02-00.jpg



frame_2025-05-14_09-02-01.jpg



frame_2025-05-14_09-02-02.jpg



frame_2025-05-14_09-02-03.jpg



frame_2025-05-14_09-02-04.jpg



frame_2025-05-14_09-02-05.jpg



frame_2025-05-14_09-02-06.jpg



frame_2025-05-14_09-02-07.jpg



frame_2025-05-14_09-02-08.jpg



frame_2025-05-14_09-02-09.jpg



frame_2025-05-14_09-02-10.jpg



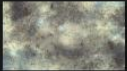
frame_2025-05-14_09-02-11.jpg



frame_2025-05-14_09-02-12.jpg



frame_2025-05-14_09-02-13.jpg



frame_2025-05-14_09-02-14.jpg



frame_2025-05-14_09-02-15.jpg



frame_2025-05-14_09-02-16.jpg



frame_2025-05-14_09-02-17.jpg



frame_2025-05-14_09-02-18.jpg



frame_2025-05-14_09-02-19.jpg



frame_2025-05-14_09-02-20.jpg



frame_2025-05-14_09-02-21.jpg



frame_2025-05-14_09-02-22.jpg



frame_2025-05-14_09-02-23.jpg



frame_2025-05-14_09-02-24.jpg



frame_2025-05-14_09-02-25.jpg



frame_2025-05-14_09-02-26.jpg



frame_2025-05-14_09-02-27.jpg



frame_2025-05-14_09-02-28.jpg



frame_2025-05-14_09-02-29.jpg



frame_2025-05-14_09-02-30.jpg



frame_2025-05-14_09-02-31.jpg



frame_2025-05-14_09-02-32.jpg



frame_2025-05-14_09-02-33.jpg



frame_2025-05-14_09-02-34.jpg



frame_2025-05-14_09-02-35.jpg



frame_2025-05-14_09-02-36.jpg



frame_2025-05-14_09-02-37.jpg



frame_2025-05-14_09-02-38.jpg



frame_2025-05-14_09-02-39.jpg



frame_2025-05-14_09-02-40.jpg



frame_2025-05-14_09-02-41.jpg



frame_2025-05-14_09-02-42.jpg



frame_2025-05-14_09-02-43.jpg



frame_2025-05-14_09-02-44.jpg



frame_2025-05-14_09-02-45.jpg



frame_2025-05-14_09-02-46.jpg



frame_2025-05-14_09-02-47.jpg



frame_2025-05-14_09-02-48.jpg

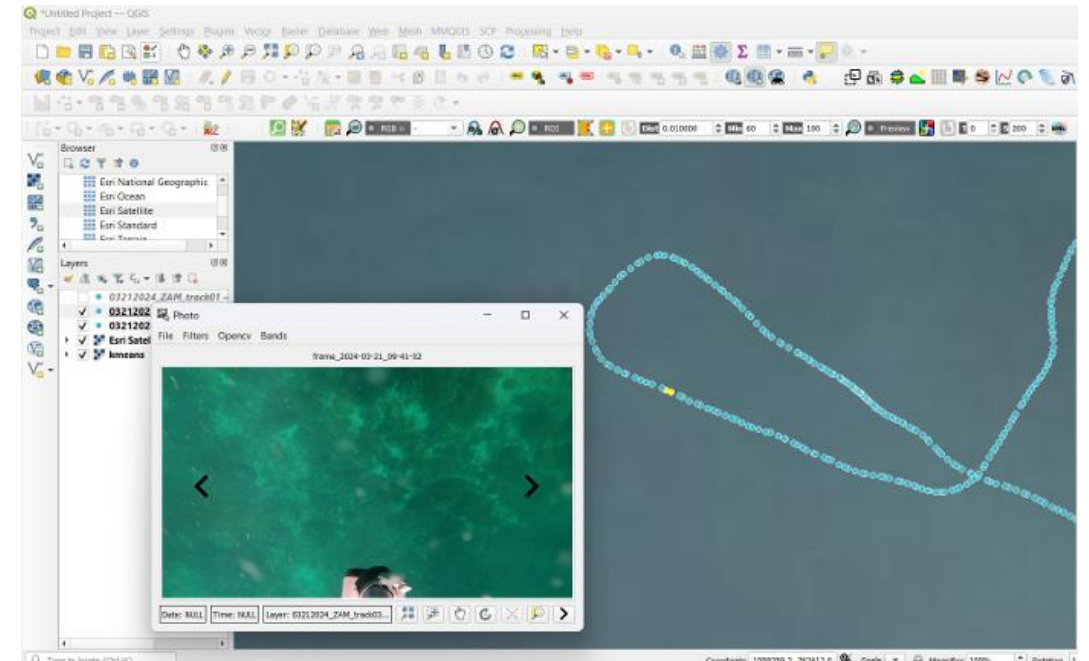
EXTRACTED FRAMES



GEOTAGGING

```
Code Blame 147 lines (119 loc) · 5.15 KB
1  import os
2  import cv2
3  import time
4  import glob
5  import pytz
6  import subprocess
7
8  import numpy as np
9  import pandas as pd
10
11  import xml.etree.ElementTree as ET
12
13  import PIL
14  from PIL import Image
15  from PIL import PngImagePlugin
16  from datetime import datetime, timedelta, timezone
17
18  def convert_utc_to_pst(utc_time_str):
19      utc_time = datetime.strptime(utc_time_str, '%Y-%m-%dT%H:%M:%SZ')
20      utc_zone = pytz.timezone('UTC')
21      pst_zone = pytz.timezone('Asia/Manila')
22      utc_time = utc_zone.localize(utc_time)
23      pst_time = utc_time.astimezone(pst_zone)
24      return pst_time.strftime('%Y-%m-%dT%H:%M:%SZ')
25
26  def read_gpx(gpx_file):
27      tree = ET.parse(gpx_file)
28      root = tree.getroot()
29
30      namespace = {
31          'default': 'http://www.topografix.com/GPX/1/1',
32          'gpxtpx': 'http://www.garmin.com/xmlschemas/TrackPointExtension/v1',
```

Extracted frames are then geotagged using the bathymetry data from echosounder.





CLASSIFICATION

```
# Memory efficient
torch.backends.cudnn.benchmark = True
torch.set_float32_matmul_precision('medium')

# Load model
model = YOLO('/content/drive/MyDrive/BlueCARES_Seagrass/field_processing/Classification/yolov11l-benthic-classification-100epochs-v01/weights/best.pt')
model.fuse() # Fuse layers for inference speed
model.to('cuda' if torch.cuda.is_available() else 'cpu')

def get_gps_from_exif(exif_data):
    """Extract GPS coordinates from EXIF data"""
    if not exif_data:
        return None

    gps_info = None

    # Direct GPS tag
    if 34853 in exif_data:
        gps_info = exif_data[34853]

    # GPSInfo tag name
    for tag_id, value in exif_data.items():
        tag_name = PIL.ExifTags.TAGS.get(tag_id, tag_id)
        if 'gps' in str(tag_name).lower():
            gps_info = value
            break

    if not gps_info:
        return None

    # Parse GPS coordinates
    def convert_to_degrees(value):
        """Convert GPS coordinates stored in EXIF to degrees in float"""
        d = float(value[0])
        m = float(value[1])
        s = float(value[2])
        return d + (m / 60.0) + (s / 3600.0)

    gps_data = {}
```

After geotagging, the images/frames are then classified using YOLO model.



CLASSIFICATION





CLASSIFICATION





CLASSIFICATION

Photo

File Filters

macroalgae_10.jpg

macroalgae 1.00
rubble 0.00
seagrass 0.00
corals 0.00
sand 0.00

←

Date: NULL Time: NULL Layer: route

route — Features Total: 22, Filtered: 22, Selected: 1

	fid	ID	Name	Date	Time	Lon	Lat	Altitude	North
1	3	0bbf435f-3269-...	macroalgae_10...	NULL	NULL	120.614265231...	13.9041847922...	NULL	
2	9	867c272e-3c90-...	macroalgae_7.jpg	NULL	NULL	120.614242264...	13.9041897375...	NULL	
3	10	21391af8-36ac-...	macroalgae_8.jpg	NULL	NULL	120.614252910...	13.9041872229...	NULL	
4	11	a303b42f-f6c5-...	macroalgae_9.jpg	NULL	NULL	120.614260788...	13.9041856304...	NULL	
5	12	7583d942-aa97-...	rubble.jpg	NULL	NULL	120.612594215...	13.9055256451...	NULL	
6	13	35fde56a-d80e-...	rubble_1.jpg	NULL	NULL	120.612606368...	13.9055108092...	NULL	
7	14	191940d8-74d2-...	rubble_2.jpg	NULL	NULL	120.612612236...	13.9055026787...	NULL	
8	15	07b02986-2054-...	rubble_3.jpg	NULL	NULL	120.612623300...	13.9054859988...	NULL	
9	16	2e87550c-24a6-...	rubble_4.jpg	NULL	NULL	120.612633358...	13.9054688998...	NULL	
10	17	6464dac6-0b10-...	rubble_5.jpg	NULL	NULL	120.612638304...	13.9054606016...	NULL	
11	18	1c21108d-a747-...	rubble_6.jpg	NULL	NULL	120.612648948...	13.9054445921...	NULL	
12	19	058f7c17-2232-...	rubble_7.jpg	NULL	NULL	120.614274200...	13.9041828644...	NULL	
13	21	a5a6b82f-86f6-...	seagrass.jpg	NULL	NULL	120.614281576...	13.9041811879...	NULL	
14	22	5088c236-5a46-...	seagrass_1.jpg	NULL	NULL	120.614284090...	13.9041803497...	NULL	
15	1	ada349f9-eb8f-...	macroalgae.jpg	NULL	NULL	120.613796599...	13.9152998663...	NULL	
16	2	c48df41e-df12-...	macroalgae_1.jpg	NULL	NULL	120.613797185...	13.9152845275...	NULL	

Show All Features